

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0605

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO1: *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

Assessment Tool Weightage: 40%

QUESTIONS: 15

Total Marks: 25

Annexure A

APPLICATION BASED QUESTIONS

Code of Conduct:

*I **S. Sanjana (192521059)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Sanjana

SET 3

11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated.

Program:

```
# Bayes' Theorem - Oil Bottle Problem

# Probability of choosing the shops
P_X = 1 / 3      # Shop X
P_Y = 2 / 3      # Shop Y (twice as likely)

# Probability of selecting an adulterated bottle
P_A_given_X = 30 / 50 # 30 adulterated out of 50
P_A_given_Y = 20 / 60 # 20 adulterated out of 60

# Total probability of selecting an adulterated bottle
P_A = (P_A_given_X * P_X) + (P_A_given_Y * P_Y)

# Bayes' Theorem
P_X_given_A = (P_A_given_X * P_X) / P_A
P_Y_given_A = (P_A_given_Y * P_Y) / P_A

# Display results
print("Probability of selecting Shop X:", P_X)
print("Probability of selecting Shop Y:", P_Y)

print("\nProbability of adulterated bottle from Shop X:", P_A_given_X)
print("Probability of adulterated bottle from Shop Y:", P_A_given_Y)

print("\nTotal probability of getting an adulterated bottle:", round(P_A, 4))

print("\nProbability the bottle came from Shop X:", round(P_X_given_A, 4))
print("Probability the bottle came from Shop Y:", round(P_Y_given_A, 4))
```

Output:

```
Probability of selecting Shop X: 0.3333333333333333
Probability of selecting Shop Y: 0.6666666666666666

Probability of adulterated bottle from Shop X: 0.6
Probability of adulterated bottle from Shop Y: 0.3333333333333333

Total probability of getting an adulterated bottle: 0.4222

Probability the bottle came from Shop X: 0.4737
Probability the bottle came from Shop Y: 0.5263
```

12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective.

Program:

```
# Bayes' Theorem - Warehouse Section Problem

# Probability of choosing each section
P_A = 3 / 4
P_B = 1 / 4

# Probability of selecting a defective item from each section
P_D_given_A = 15 / 50
P_D_given_B = 25 / 50

# Total probability of selecting a defective item
P_D = (P_D_given_A * P_A) + (P_D_given_B * P_B)

# Bayes' Theorem
P_A_given_D = (P_D_given_A * P_A) / P_D
P_B_given_D = (P_D_given_B * P_B) / P_D

# Display the results
print("Probability of choosing Section A =", P_A)
print("Probability of choosing Section B =", P_B)
print("Probability of defective item from Section A =", P_D_given_A)
print("Probability of defective item from Section B =", P_D_given_B)
print("Total probability of selecting a defective item =", round(P_D, 4))
print("Probability that the defective item came from Section A =", round(P_A_given_D, 4))
print("Probability that the defective item came from Section B =", round(P_B_given_D, 4))
```

Output:

```
Probability of choosing Section A = 0.75
Probability of choosing Section B = 0.25
Probability of defective item from Section A = 0.3
Probability of defective item from Section B = 0.5
Total probability of selecting a defective item = 0.35
Probability that the defective item came from Section A = 0.6429
Probability that the defective item came from Section B = 0.3571
```

13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP) = 120 and True Negative (TN) = 150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. Find the following: Accuracy, Precision, Sensitivity (Recall), Specificity.

Program:

```
# Performance Metrics Calculation

# Given values
TP = 120
TN = 150

# Dataset details
samples_per_class = 1000
total_samples = samples_per_class * 2    # 2000 samples
test_samples = int(total_samples * 0.20) # 20% for testing = 400

# Total actual positives and negatives in test set
actual_positive = test_samples // 2    # 200
actual_negative = test_samples // 2    # 200

# Calculate FP and FN
FN = actual_positive - TP
FP = actual_negative - TN

# Calculate metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)

# Display results
print("True Positive (TP):", TP)
print("True Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)

print("\nPerformance Metrics")
print("Accuracy   :", round(accuracy * 100, 2), "%")
print("Precision   :", round(precision * 100, 2), "%")
print("Sensitivity :", round(recall * 100, 2), "%")
print("Specificity  :", round(specificity * 100, 2), "%")
```

Output:

```
True Positive (TP): 120
True Negative (TN): 150
False Positive (FP): 50
False Negative (FN): 80
```

```
Performance Metrics
Accuracy      : 67.5 %
Precision     : 70.59 %
Sensitivity   : 60.0 %
Specificity   : 75.0 %
```

14. A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP = 180 and TN = 160. Each class contains 1200 samples, and 25% of the dataset is used as test data. Calculate: Accuracy Precision Sensitivity Specificity.

Program:

```
# Performance Metrics Calculation

# Given values
TP = 180
TN = 160

# Dataset details
samples_per_class = 1200
total_samples = samples_per_class * 2    # 2400 samples
test_samples = int(total_samples * 0.25) # 25% for testing = 600

# Actual positives and negatives in test data
actual_positive = test_samples // 2    # 300
actual_negative = test_samples // 2    # 300

# Calculate FP and FN
FN = actual_positive - TP
FP = actual_negative - TN

# Calculate metrics
accuracy = (TP + TN) / (TP + TN + FP + FN)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)

# Display results
print("True Positive (TP):", TP)
print("True Negative (TN):", TN)
print("False Positive (FP):", FP)
print("False Negative (FN):", FN)
```

```

print("\nPerformance Metrics")
print("Accuracy   :", round(accuracy * 100, 2), "%")
print("Precision   :", round(precision * 100, 2), "%")
print("Sensitivity :", round(recall * 100, 2), "%")
print("Specificity :", round(specificity * 100, 2), "%")

```

Output:

```

True Positive (TP): 180
True Negative (TN): 160
False Positive (FP): 140
False Negative (FN): 120

Performance Metrics
Accuracy       : 56.67 %
Precision      : 56.25 %
Sensitivity    : 60.0 %
Specificity    : 53.33 %

```

15. A deep learning model is trained for a classification task, and the following loss values are observed:

- i. Epoch 1: Training Loss = 2.8, Validation Loss = 2.6**
- ii. Epoch 2: Training Loss = 2.4, Validation Loss = 2.2**
- iii. Epoch 3: Training Loss = 2.1, Validation Loss = 1.9**
- iv. Epoch 4: Training Loss = 1.9, Validation Loss = 1.8**
- v. Epoch 5: Training Loss = 1.7, Validation Loss = 1.9**
- vi. Epoch 6: Training Loss = 1.5, Validation Loss = 2.1**
- vii. Epoch 7: Training Loss = 1.3, Validation Loss = 2.4**
- viii. Epoch 8: Training Loss = 1.2, Validation Loss = 2.7**

Answer the following:

- a. At which epoch does the model achieve the best generalization performance?**
- b. At which point does overfitting begin? Justify your answer.**
- c. Suggest two techniques to reduce overfitting and briefly explain how they help.**

Program:

```

import matplotlib.pyplot as plt

# Epoch numbers
epochs = [1, 2, 3, 4, 5, 6, 7, 8]

# Training and Validation Loss
training_loss = [2.8, 2.4, 2.1, 1.9, 1.7, 1.5, 1.3, 1.2]
validation_loss = [2.6, 2.2, 1.9, 1.8, 1.9, 2.1, 2.4, 2.7]

```

```

# Find the best epoch (minimum validation loss)
best_epoch = validation_loss.index(min(validation_loss)) + 1

# Overfitting starts after the best epoch
overfitting_epoch = best_epoch + 1

# Print answers
print("----- RESULTS -----")
print("a. Best Generalization Performance : Epoch", best_epoch)
print("  Validation Loss =", min(validation_loss))

print("\nb. Overfitting Begins : Epoch", overfitting_epoch)
print("  Reason: Validation loss increases while Training loss continues to decrease.")

print("\nc. Techniques to Reduce Overfitting:")
print("  1. Dropout - Randomly deactivates neurons during training.")
print("  2. Early Stopping - Stops training when validation loss starts increasing.")

# Plot the graph
plt.figure(figsize=(8,5))
plt.plot(epochs, training_loss, marker='o', label='Training Loss')
plt.plot(epochs, validation_loss, marker='s', label='Validation Loss')

# Highlight best epoch
plt.scatter(best_epoch, min(validation_loss), color='red', s=100, label='Best Generalization')

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training Loss vs Validation Loss")
plt.xticks(epochs)
plt.legend()
plt.grid(True)

plt.show()

```

Output:

```

a. Best Generalization Performance : Epoch 4
   Validation Loss = 1.8

b. Overfitting Begins : Epoch 5
   Reason: Validation loss increases while Training loss continues to decrease.

c. Techniques to Reduce Overfitting:
   1. Dropout - Randomly deactivates neurons during training.
   2. Early Stopping - Stops training when validation loss starts increasing.

```

